

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning
(BL 5)

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	30%	15%	20%	20%	15%	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks :

Annexure A

Short Answer Test

1. Compare **Linear Regression and Polynomial Regression** for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.
2. Compare **Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM)** networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient problem, computational complexity, and prediction accuracy.
3. Compare **Naïve Bayes and Support Vector Machines (SVM)** for classification problems such as sentiment analysis. Analyze differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data. Discuss scenarios where each method is preferable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

COMPARATIVE ANALYSIS OF MACHINE LEARNING ALGORITHMS

Linear Regression vs Polynomial Regression | RNN vs LSTM | Naïve Bayes vs SVM

1. Linear Regression vs Polynomial Regression for Housing Price Prediction
Linear Regression and Polynomial Regression are supervised learning techniques that can be used to predict continuous values such as housing prices. Linear Regression assumes that the relationship between input variables and price can be represented approximately by a straight line, whereas Polynomial Regression introduces higher-order terms to capture curved and non-linear relationships.

Factor	Linear Regression	Polynomial Regression
Basic idea	Models a straight-line relationship between input features and house price.	Extends linear regression by adding powers such as x^2 , x^3 , etc.
Model complexity	Low complexity and simple structure.	Higher complexity; increases with polynomial degree.
Non-linear relationships	Poor at representing strongly non-linear relationships.	Can capture curved and non-linear relationships.
Overfitting risk	Generally low.	Higher, especially with high polynomial degrees.
Interpretability	Very easy to understand and explain.	Less intuitive because higher-order coefficients are harder to interpret.
Training time	Fast.	Usually slower as polynomial features increase.
Generalization	Often good when the relationship is approximately linear.	Good when the true relationship is non-linear and degree is selected carefully.

Example

For house area as an input feature, Linear Regression can be represented as:

$$\text{Price} = \beta_0 + \beta_1(\text{Area})$$

Polynomial Regression can be represented as:

$$\text{Price} = \beta_0 + \beta_1(\text{Area}) + \beta_2(\text{Area}^2) + \beta_3(\text{Area}^3)$$

If house prices increase approximately proportionally with area, Linear Regression is suitable. If the price relationship changes in a curved manner—for example, prices rise rapidly for medium-sized houses and then level off—Polynomial Regression can provide a better fit.

Overfitting

Polynomial Regression becomes increasingly flexible as the degree increases. A low-degree polynomial such as degree 2 or 3 can capture useful curvature. However, a very high-degree polynomial may fit random noise in the training data. This produces very low training error but poor performance on unseen houses. Validation or cross-validation should therefore be used to select an appropriate polynomial degree.

When Each Model Performs Better

Linear Regression is better when:

The relationship between features and price is approximately linear.

The dataset is relatively small.

Interpretability is important.

Fast training and prediction are required.

There is little evidence of strong non-linearity.

Polynomial Regression is better when:

Housing prices have a clear curved relationship with one or more features.

Linear Regression underfits the data.

There is enough data to support additional model complexity.

The polynomial degree can be selected carefully using validation.

Conclusion: Linear Regression is simpler, faster, and more interpretable, while Polynomial Regression provides greater flexibility for non-linear housing-price relationships but introduces greater complexity and overfitting risk.

2. RNN vs LSTM for Time-Series Prediction

Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are designed to process sequential data. They can be used for stock-price forecasting, weather prediction, sensor monitoring, demand forecasting, and other time-series applications.

Factor	RNN	LSTM
Architecture	Simple recurrent structure with a hidden state.	Uses memory cells and input, forget, and output gates.
Long-term dependencies	Difficult to retain information over long sequences.	Designed to retain important information over long periods.
Vanishing gradient	Strongly affected during training.	Significantly reduces the vanishing-gradient problem.
Computational complexity	Lower.	Higher because of multiple gates.

Memory capability	Relatively short-term memory.	Long-term memory capability.
Training speed	Usually faster.	Usually slower than a basic RNN.
Parameters	Fewer parameters.	More parameters.
Prediction accuracy	Can perform well for short/simple sequences.	Often better for complex sequences with long-term dependencies.
Implementation	Simpler.	More complicated.

RNN

A basic RNN updates its hidden state using the current input and the previous hidden state:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

The hidden state carries information from previous time steps. However, when sequences become long, gradients can become extremely small during backpropagation through time. This is known as the vanishing gradient problem, and it can cause the network to forget information from earlier time steps.

LSTM

LSTM introduces a memory cell and three important gates. The forget gate determines which old information should be discarded, the input gate determines which new information should be stored, and the output gate determines which information should be passed forward. This mechanism allows an LSTM to selectively retain important information over long periods.

Example: Stock Price Forecasting

Suppose the goal is to predict tomorrow's stock price using the previous 100 trading days. A basic RNN may have difficulty retaining information from the beginning of this sequence. An LSTM can preserve important information through its cell state and gates. However, stock prices are noisy and influenced by many external factors, so LSTM does not automatically guarantee better forecasts. Data quality, feature selection, validation, and avoiding data leakage remain important.

Computational Complexity

RNNs perform fewer operations per time step because their architecture is simpler. LSTMs perform additional calculations for their gates, increasing computational cost and the number of parameters. Therefore, LSTMs normally require more memory and training time than basic RNNs.

When Each Model Performs Better

RNN is preferable when:

- The sequence is relatively short.

- Dependencies are mostly recent.

- Computational resources are limited.

- A simpler model is desired.

- Fast training is important.

LSTM is preferable when:

The sequence is long.

Long-term dependencies are important.

The problem suffers from the limitations of basic RNNs.

More computational resources are available.

Higher predictive performance justifies increased complexity.

Conclusion: RNNs are simpler and computationally cheaper, while LSTMs are better suited to long-term dependencies because their memory-cell and gating mechanism reduces the vanishing-gradient problem.

3. Naïve Bayes vs SVM for Sentiment Analysis

Naïve Bayes and Support Vector Machines (SVM) are widely used classification algorithms for text classification and sentiment analysis. Naïve Bayes is probability-based, while SVM searches for a decision boundary that maximizes the margin between classes.

Factor	Naïve Bayes	SVM
Basic principle	Uses Bayes' theorem to calculate class probabilities.	Finds a decision boundary that maximizes the margin between classes.
Main assumption	Features are conditionally independent given the class.	Does not require feature independence.
Decision boundary	Based on estimated probabilities.	Explicit maximum-margin decision boundary.
Training speed	Very fast.	Usually slower than Naïve Bayes.
Prediction speed	Very fast.	Fast, depending on support vectors.
High-dimensional data	Performs well.	Performs particularly well with high-dimensional sparse data.
Interpretability	Relatively easy to understand.	More difficult, especially with non-linear kernels.
Computational requirements	Low.	Higher, especially with complex kernels.
Non-linear classification	Limited in its basic form.	Can model non-linear boundaries using kernels.

Naïve Bayes

Naïve Bayes is based on Bayes' theorem:

$$P(C|X) = P(X|C)P(C) / P(X)$$

For sentiment analysis, the classes may be Positive, Negative, and Neutral. The model estimates the probability that a document belongs to each class based on its words. Words such as 'excellent' and 'wonderful' may increase the probability of positive sentiment, while words such as 'bad' and 'terrible' may increase the probability of negative sentiment.

The naïve assumption is that features are conditionally independent given the class. Although this assumption is not completely true for natural language, Naïve Bayes can still perform well in many text-classification tasks.

SVM

SVM attempts to find a decision boundary that separates classes while maximizing the margin between them. A simplified decision function is:

$$f(x) = w^T x + b$$

SVM is particularly effective for text classification because representations such as TF-IDF can contain thousands or millions of dimensions while individual documents are usually sparse.

Decision Boundaries

Naïve Bayes creates a classification rule based on probability distributions.

SVM directly searches for an optimal separating hyperplane. In simple terms, Naïve Bayes asks which sentiment has the highest probability for the observed words, while SVM asks which boundary best separates positive and negative examples.

Computational Efficiency

Naïve Bayes is usually extremely fast to train because it mainly calculates feature and class statistics. SVM can require more computational resources, particularly for large datasets or when non-linear kernels are used. For very large sentiment-analysis datasets where training speed is important, Naïve Bayes can therefore be an attractive baseline.

High-Dimensional Data

Both methods can perform well with high-dimensional text data. A document represented using TF-IDF might have thousands or more features. Naïve Bayes can process these features efficiently because its calculations are relatively simple. SVM is also well suited to this situation because it can construct a strong separating boundary in a high-dimensional feature space.

When Each Model Performs Better

Naïve Bayes is preferable when:

- Training speed is important.

- The dataset is small or moderately sized.

- A simple baseline model is needed.

- Features are word counts or TF-IDF values.

- Computational resources are limited.

- Rapid model development is required.

SVM is preferable when:

Classification accuracy is a priority.

The dataset contains high-dimensional text features.

There is sufficient training data.

A strong separating boundary is possible.

More computational resources are available.

Complex classification relationships need to be modeled using suitable kernels.

Overall Comparison

Algorithm Pair	Simpler Model	Better for Complex Relationships	Main Risk/Disadvantage	Typical Strong Use Case
Linear vs Polynomial Regression	Linear Regression	Polynomial Regression	Polynomial overfitting	Housing-price prediction
RNN vs LSTM	RNN	LSTM	LSTM has greater computational complexity	Long-term time-series forecasting
Naïve Bayes vs SVM	Naïve Bayes	SVM	SVM can be computationally expensive	Text/sentiment classification

Final Conclusion

The choice of machine-learning algorithm depends on the characteristics of the problem. Linear Regression is preferable when housing prices have approximately linear relationships, whereas Polynomial Regression is useful when significant non-linear patterns exist. For sequential data, LSTM generally has an advantage over basic RNNs when long-term dependencies are important, although this comes with increased computational cost. For sentiment analysis, Naïve Bayes offers simplicity and very fast computation, while SVM can provide strong classification performance for high-dimensional text data. In practice, model selection should consider validation performance, computational requirements, interpretability, dataset size, and the nature of the underlying data.